
بسم الله الرحمن الرحيم

نام درس: آزمایشگاه ریزپردازنده

نام استاد: دکتر امین یوسفی

گزارش پروژه قفل دیجیتال

نام و نام خانوادگی اعضا:

بیتا اخگر ۴۰۱۴۰۵۰۰۴

کیمیا مشهدی زاده ۴۰۰۴۰۵۰۷۱

دریا یزدان پناه ۴۰۱۴۰۵۰۶۵

نیمسال تحصیلی اول ۱۴۰۵- ۱۴۰۴

فهرست مطالب

شرح پروژه ۴

پیاده سازی سخت افزاری ۵

کد پروژه ۶

شرح پروژه

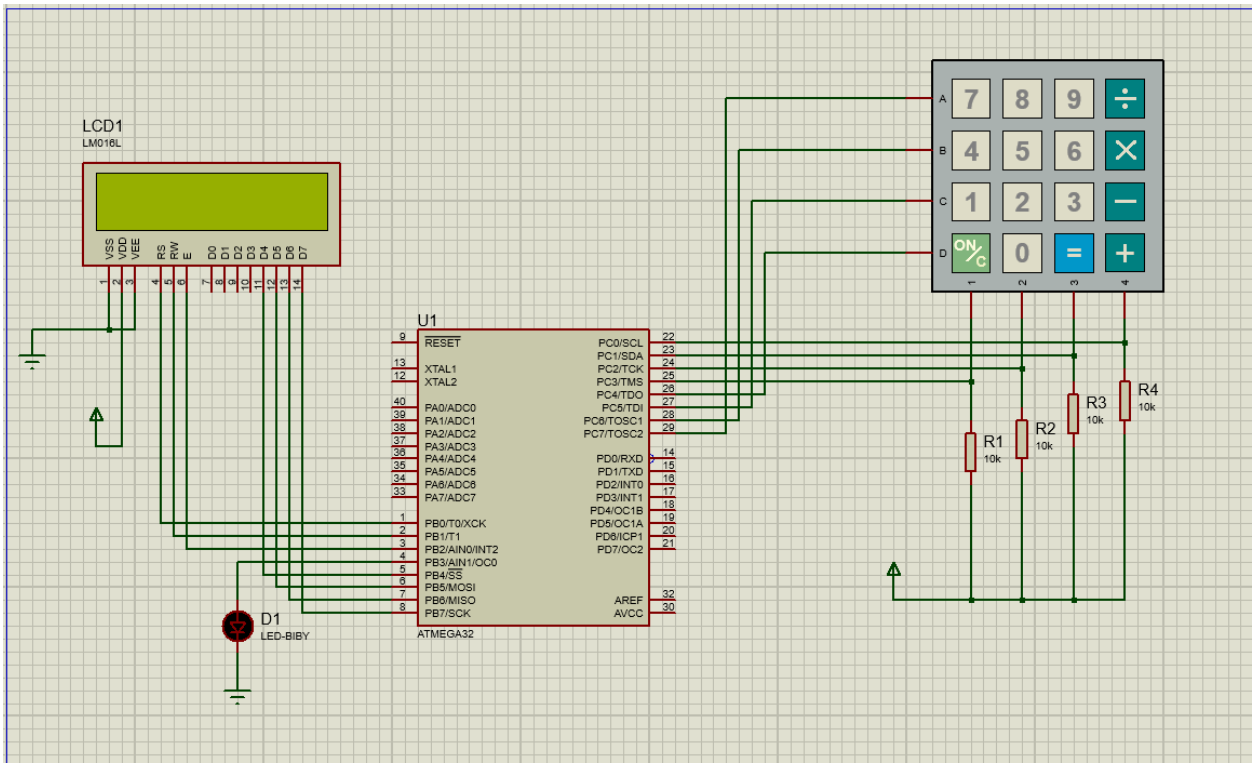
در این پروژه یک قفل دیجیتال را در محیط Proteus با استفاده از میکروکنترلر AVR ، صفحه کلید ماتریسی و صفحه نمایش LCD پیاده سازی کردیم. با اجرای برنامه از کاربر رمز عبور روی LCD پرسیده می‌شود، در صورت وارد کردن رمز صحیح (که از قبل 1234 در نظر گرفته شده است) با استفاده از صفحه کلید و سپس زدن کلید = ، از کاربر پرسیده می‌شود که آیا می‌خواهد رمز عبور را تغییر دهد یا خیر؟ کاربر با وارد کردن ۱ (بله) و یا صفر (خیر) می‌تواند روند ادامه را تعیین کند. با انتخاب کردن ۱، از کاربر رمز جدید پرسیده می‌شود که کاربر بعد از وارد کردن رمز و زدن کلید = رمز را تغییر می‌دهد. اگر صفر انتخاب شود، اتفاقی نمی‌افتد و سیستم دوباره قفل می‌شود تا وقتی که رمز صحیح قبلی دوباره وارد شود.

کاربر ۳ بار فرصت دارد تا رمز صحیح را وارد کند. هر بار که رمز اشتباه را وارد کند تعداد تلاش ها روی نمایشگر نشان داده می‌شود. در صورتی که ۳ بار رمز اشتباه وارد شود، LED پنج بار چشمک می‌زند و سپس شمارش معکوس از ۶۰ شروع می‌شود و متناسب با شمارش، LED نیز چشمک می‌زند. با اتمام ۶۰ ثانیه کاربر دوباره می‌تواند رمز را وارد کند.

پیاده سازی سخت افزاری

سخت افزار این سیستم متشکل است از:

- میکروکنترلر ATmega32
- LM016L LCD
- صفحه کلید ماتریسی
- LED زرد
- مقاومت 10k



کد پروژه

ابتدا کتابخانه های مورد نیاز را include می‌کنیم. سپس ماتریس معادل صفحه کلید و بقیه متغیرهای مورد استفاده در کد را تعریف می‌کنیم.

```
#include <mega32.h>
#include <stdio.h>
#include <delay.h>
#include <string.h>
#include <stdint.h>
#include <stdbool.h>

#asm
    .equ __lcd_port=0x18 ; PORTB
#endasm
#include <lcd.h>

// keypad matrix
unsigned char code[4][4] = {
    {7, 4, 1, 10},
    {8, 5, 2, 0},
    {9, 6, 3, 11},
    {12,13,14,15}
};

// variables
char pass[] = "1234";      // password
char pl[5] = "";
uint8_t pl_len = 0;
uint8_t att = 0;
uint8_t max_attempts = 3;
bool in_change_pass_mode = false;

```

تابع delay

در ادامه تابعی برای ایجاد تاخیر آمده است که به اندازه‌ی پارامتر time ضربدر 0.01 ثانیه تاخیر تولید می‌کند. ابتدا TCCR0 را برابر با 0x05 قرار دادیم تا پالس ساعت را روی ۱۰۲۴ بگذاریم. سپس با استفاده از معادلات زیر به عدد ۲۴۶ برای TCNT0 رسیدیم.

اگر زمان مورد نظر را ۰/۰۱ ثانیه، پالس ساعت را ۱۰۲۴ و فرکانس را 1 MHz در نظر بگیریم، داریم:

$$\frac{1}{100} = \frac{1024(1+x)}{1\,MHz} = 9.7 \approx 10 \rightarrow 1 + x = 10 \rightarrow x = 9$$

$$255 - 9 = 246 \rightarrow TCNT0 = 246$$

فلگ TOV0 با رسیدن TCNT0 به 0xFF فعال می‌شود پس با انتساب ۲۴۶ به TCNT0، می‌دانیم ۰/۰۱ ثانیه طول می‌کشد تا فلگ TOV0 فعال شود.

```
28
29
30  void delay(int time) {
31      unsigned char i = 0;
32      TCCR0 = 0x05;
33      TCNT0 = 246;
34
35      while (i < time) {
36          while ((TIFR & 0x01) == 0) {}
37          TIFR = 0x01;
38          TCNT0 = 246;
39          i++;
40      }
41  }
42
```

سپس دو حلقه‌ی تو در تو داریم که حلقه‌ی اول به اندازه time تکرار می‌شود و در داخل آن حلقه‌ی دیگری داریم که شرط خارج شدن از آن حلقه فعال شدن فلگ TOV0 است که یعنی مقدار رجیستر TIFR برابر با 0x01 شده است. این پروسه همانطور که توضیح داده شد یک صدم ثانیه زمان می‌برد و پس از آن رجیستر TIFR ریست می‌شود و TCNT0 همان مقدار اولیه‌ی ۲۴۶ قرار داده می‌شود تا این روند دوباره تکرار شود تا زمانی که i برابر با time شود. وقتی از این حلقه خارج می‌شویم یعنی تاخیر time x 0.01s طی شده است.

تابع led_flash_warning

تابع بعدی مربوط به چشمک زدن LED می‌باشد. در حلقه‌ای که ۵ بار تکرار می‌شود، ابتدا LED روشن می‌شود و پس از تاخیر ۰/۲ ثانیه‌ای LED خاموش می‌شود و با یک تاخیر دیگر حلقه به پایان می‌رسد. در نتیجه LED ۵ بار روشن و خاموش می‌شود (چشمک می‌زند).

```
43  // LED warning flash
44  void led_warning_flash(void) {
45      uint8_t i;
46
47      for (i = 0; i < 5; i++) {
48          PORTB |= (1 << 3);    // LED ON
49          delay(20);
50          PORTB &= ~(1 << 3);   // LED OFF
51          delay(20);
52      }
53  }
54
```

تابع timer

تابع بعدی مربوط به زمانی است که کاربر رمز عبور را ۳ بار اشتباه زده است و حالا شمارش معکوس و LED چشمک زدن شروع به کار کرده‌اند. برای ایجاد تاخیر ۰/۲۵ ثانیه‌ای، معادلات زیر را حل کرده تا به عدد مناسب برای TCNT0 برسیم.

$$\frac{25}{100} = \frac{1024(1+x)}{1\ MHz} \rightarrow x = 243$$

$$255 - 243 = 12 \rightarrow\ TCNT0 = 12$$

با توجه به معادله‌ی بالا و توضیحات مربوط به تابع delay می‌دانیم که حالا با مساوی قرار دادن TCNT0 می‌توانیم در ادامه تاخیر ۰/۲۵ ثانیه‌ای تولید کنیم. در حلقه‌ی while که به اندازه‌ی time تکرار می‌شود، ابتدا زمان باقی مانده را روی LCD نمایش می‌دهیم و LED را روشن می‌کنیم، سپس یه تاخیر نیم ثانیه‌ای ایجاد می‌کنیم که در واقع این تاخیر با دو بار تکرار تاخیر ۰/۲۵ ثانیه‌ای تولید می‌شود. با گذشت نیم ثانیه، LED خاموش می‌شود و یک تاخیر نیم ثانیه‌ای دیگر خواهیم داشت تا یک ثانیه کامل بگذرد. این روند به اندازه time تکرار می‌شود که در ادامه می‌بینیم time برابر با ۶۰ قرار داده شده است. در نتیجه روی LCD اعداد ۶۰ تا صفر نمایش داده می‌شوند و به ازای هر عدد یک بار LED روشن و خاموش می‌شود، کل این پروسه ۶۰ ثانیه زمان می‌برد.

```
54
55  // timer with LED flash warning
56  void timer(int time) {
57      unsigned char i = 0;
58      unsigned char j;
59      char s[3];
60
61      TCCR0 = 0x05;
62      TCNT0 = 12;
63      lcd_clear();
64      lcd_puts("SYSTEM LOCKED!");
65      lcd_gotoxy(0, 1);
66      lcd_puts("Wait: ");
67
68      while (i < time) {
69          // update countdown and turn LED on
70          sprintf(s, "%u", (unsigned)(time - i));
71          lcd_gotoxy(6, 1);
72          lcd_puts(s);
73          lcd_puts("s  ");
74          PORTB |= (1 << 3);  // LED on
75
76          // wait 0.5 second
77          for (j = 0; j < 2; j++) {
78              while ((TIFR & 0x01) == 0) {}
79              TIFR = 0x01;
80              TCNT0 = 12;
81          }
83          // Turn LED OFF
84          PORTB &= ~(1 << 3);
85
86          // wait another 0.5 second
87          for (j = 0; j < 2; j++) {
88              while ((TIFR & 0x01) == 0) {}
89              TIFR = 0x01;
90              TCNT0 = 12;
91          }
92
93          i++;
94      }
95
96      // turn off LED
97      PORTB &= ~(1 << 3);
98      lcd_clear();
99      lcd_gotoxy(0, 0);
100     lcd_puts("You can retry");
101     delay(150);
102 }
103
```

تابع scan_key

این تابع کلید فشار داده‌شده از صفحه کلید ماتریسی را با روش اسکن ستون، به همراه فعال‌سازی ستون‌ها از طریق PORTC و خواندن ورودی‌ها از PINC تشخیص می‌دهد و مقدار خروجی آن مستقیماً از آرایه code استخراج می‌شود. اگر کلیدی فشار داده نشده یا نامشخص باشد، خروجی برابر با 0xFF است و در غیراین صورت مقدار کلید برابر با یکی از عناصر code[0..3][0..3] در ابتدا دیدیم است که به ازای هر ستون i با دیتای ورودی 0x07، 0x0B، 0x0D یا 0x0E مطابقت داده می‌شود و نتیجه برگردانده می‌شود.

```
104
105 unsigned char scan_key(void) {
106     unsigned char i, data, num_key, temp;
107     num_key = 0xFF;
108     temp = 0x70;
109
110     for (i = 0; i < 4; i++) {
111         PORTC = temp;
112         delay(1);
113         data = PINC & 0x0F;
114
115         if (data == 0x07) {num_key = code[0][i]; break;}
116         if (data == 0x0B) {num_key = code[1][i]; break;}
117         if (data == 0x0D) {num_key = code[2][i]; break;}
118         if (data == 0x0E) {num_key = code[3][i]; break;}
119
120         temp = ((temp >> 1) | 0x80) & 0xF0;
121     }
122     return num_key;
123 }
124
```

تابع display_input

این تابع برای نمایش اعداد رمز ورودی به صورت * روی LCD می‌باشد. عددی که کاربر وارد می‌کند در آرایه‌ی pl ذخیره می‌شود و pl_len نیز طول این آرایه است. با فراخوانی این تابع به اندازه طول رمزی که وارد شده ستاره نمایش داده می‌شود. همچنین اگر کاربر در حال وارد کردن رمز جدید باشد قبل از عدد وارد شده New Pass: نوشته و در غیر این صورت Pass: نوشته می‌شود. تفاوت این دو حالت با true یا false بودن بولین in_change_pass_mode مشخص می‌شود.

```
125
126 void display_input(void) {
127     uint8_t i;
128     lcd_clear();
129
130     if (in_change_pass_mode) {
131         lcd_puts("New Pass: ");
132     } else {
133         lcd_puts("Pass: ");
134     }
135
136     for (i = 0; i < pl_len; i++) {
137         lcd_putchar('*');
138     }
139 }
```

تابع اصلی main

در ابتدا input یا output بودن پورت ها را مشخص می‌کنیم. پورت B را که LCD و LED به آن متصل است خروجی قرار می‌دهیم و با 0x0F قرار دادن پورت C، PC0:3 را ورودی و PC4:7 را خروجی قرار می‌دهیم زیرا ستون های صفحه کلید به PC0:3 و سطر های آن به PC4:7 متصل هستند. سپس LCD را initialize می‌کنیم و جملات Digital Lock و State: LOCKED را روی آن نمایش می‌دهیم. بعد از یک تاخیر دو ثانیه‌ای :Enter Password را نمایش می‌دهیم.

```
141
142 void main(void) {
143     uint8_t result;
144
145     DDRB = 0xFF;
146     DDRC = 0xF0;
147     PORTC = 0x0F; // Pull-up for columns
148
149
150     lcd_init(16);
151     lcd_clear();
152     lcd_gotoxy(0, 0);
153     lcd_puts("Digital Lock");
154     lcd_gotoxy(0, 1);
155     lcd_puts("State: LOCKED");
156     delay(200);
157
158     lcd_clear();
159     lcd_puts("Enter Password:");
160
```

در حلقه‌ی while تابع main، ابتدا کلید فشرده شده را از طریق فراخوانی تابع scan_key در متغیر result ذخیره می‌کنیم. در صورتی که result برابر با ۲۵۵ نباشد یعنی یک کلید معتبر فشرده شده است. سپس با بررسی بولین in_change_pass_mode بررسی می‌کنیم که آیا کاربر در حال تغییر رمز عبور هست یا خیر.

کد زیر مربوط به موقعیتی است که کاربر در حال تغییر رمز است و کلید = را فشرده است. در این صورت بررسی می‌شود آیا طول رمز وارد شده ۴ می‌باشد یا خیر. اگر ۴ بود آن را در pass کپی می‌کند و بولین in_change_pass_mode را false قرار می‌دهد زیرا پروسه‌ی تغییر رمز عبور به پایان رسیده است. رمز جدید کاربر باید حتما ۴ عدد باشد و اگر اینگونه نباشد و کاربر = را فشرده باشد، رمز جدیدی ذخیره نمیشود.

```
161     while (1) {
162         result = scan_key();
163 
164         if (result != 255) {
165 
166             // user is changing password
167             if (in_change_pass_mode) {
168                 if (result == 11) {  // Enter
169                     if (pl_len == 4) {
170                         strcpy(pass, pl);
171                         lcd_clear();
172                         lcd_puts("Pass Changed!");
173                         delay(150);
174 
175                         in_change_pass_mode = false;
176                         lcd_clear();
177                         lcd_puts("Enter Password:");
178                     }
179                     pl_len = 0;
180                     pl[0] = '\0';
181                 }
```

قسمت های بعدی شرایطی را نشان می‌دهند که کاربر در حال تغییر تغییر رمز است ولی یکی از کلید های – یا on/c را فشرده یا اینکه هنوز رمز را به طور کامل وارد نکرده است. کلید on/c تمام اعداد وارد شده را حذف می‌کند، طول pl را صفر قرار می‌دهد و آرایه را خالی می‌کند. کلید – آخرین عدد وارد شده را پاک می‌کند.

اگر کاربر هنوز در حال وارد کردن اعداد رمز باشد و طول رمز به ۴ نرسیده باشد، هر عدد به آرایه pl اضافه می‌شود و یکی به طول آن اضافه می‌شود و رمز با تابع display_input نمایش داده می‌شود.

```
            }
            else if (result == 10) {  // on/c - Clear all
                pl_len = 0;
                pl[0] = '\0';
                display_input();
            }
            else if (result == 14) {  // C - Backspace (delete last digit)
                if (pl_len > 0) {
                    pl_len--;
                    pl[pl_len] = '\0';
                }
                display_input();
            }
            else if (result <= 9 && pl_len < 4) {
                pl[pl_len++] = '0' + result;
                pl[pl_len] = '\0';
                display_input();
            }

            delay(30);
            continue;
        }
```

این قسمت وضعیتی را نشان می‌دهد که کاربر = را فشرده و در حال تغییر رمز نمی‌باشد، مانند زمانی که تازه برنامه اجرا شده و کاربر برای اولین بار باید رمز را وارد کند. در این قسمت رمز وارد شده با رمز ذخیره در pass که می‌تواند رمز از پیش تعیین شده یا رمزی باشد که کاربر تغییر داده و ذخیره کرده باشد، مقایسه می‌شود، اگر این دو برابر باشند، کاربر ورود کرده و از او پرسیده می‌شود که آیا میخواهد رمز را تغییر دهد یا خیر، جواب کاربر در مراحل جلوتر بررسی می‌شود.

```

        // normal
        if (result == 11) {  // Enter key
            // Check Password
            if (strcmp(pl, pass) == 0) {
                lcd_clear();
                lcd_puts("Access Granted!");
                delay(200);

                // Ask for password change
                lcd_clear();
                lcd_puts("Change Pass?");
                lcd_gotoxy(0, 1);
                lcd_puts("1=Yes  0=No");
```

در اینجا در دو جواب yes که با وارد کردن ۱ و جواب no که با صفر مشخص می‌شود بررسی می‌شود. اگر کاربر ۱ را فشـرده باشـد، بولین in_change_pass_mode، true می‌شـود، آرایه‌ی pl خالی می‌شـود و از حلقه خارج می‌شـود و وارد قسـمتی میشـود که شـرط in_change_pass_mode= true برقرار بود که بالا تر توضـیح داده شـده. اگر کاربر صـفر را انتخاب کرده باشـد، :Enter Password نمایش داده شـده، از حلقه خارج شده، و روند وارد کردن رمز عبور و غیره مانند ابتدای برنامه دوباره تکرار می‌شود.

```
217
218                 // Wait for response
219                 while(1) {
220                     result = scan_key();
221                     if (result == 1) {
222                         // Enter change password mode
223                         in_change_pass_mode = true;
224                         lcd_clear();
225                         lcd_puts("Enter New Pass:");
226                         pl_len = 0;
227                         pl[0] = '\0';
228                         delay(200);
229                         break;
230                     }
231                     else if (result == 0) {
232                         // Exit
233                         lcd_clear();
234                         lcd_puts("Enter Password:");
235                         break;
236                     }
237                     delay(1);
238                 }
239
240                 att = 0;
241                 pl_len = 0;
242                 pl[0] = '\0';
243             }
```

حال شرایطی بررسی می‌شود که کاربر رمز را اشتباه وارد کرده است. اگر تعداد تلاش های اشتباه کاربر به ۳ تا نرسیده باشد، یک واحد به متغیر att که برای شمارش تعداد دفعاتی است که کاربر رمز را اشتباه وارد کرده است، اضافه می‌شود. و روی LCD نمایش داده می‌شود. اگر تعداد تلاش ها به ۳ رسیده باشد، با فراخوانی led_flash_warning ، LED ۵ بار چشمک می‌زند و همچنین تابع timer نیز فراخوانی می‌شود که کاربرد آن توضیح داده شده است.

```
244                 // Wrong Password
245                 else {
246                     pl_len = 0;
247                     pl[0] = '\0';
248                     att++;
249 
250                     if (att >= max_attempts) {
251                         led_warning_flash();  // LED warning flash
252                         timer(60);            // Lock for 60 seconds with LED blink
253                         att = 0;
254 
255                         lcd_clear();
256                         lcd_puts("Enter Password:");
257                     } else {
258                         lcd_clear();
259                         lcd_puts("Wrong Password!");
260                         lcd_gotoxy(0, 1);
261                         lcd_puts("Attempts: ");
262                         lcd_putchar('0' + att);
263                         lcd_putchar('/');
264                         lcd_putchar('0' + max_attempts);
265                         delay(200);
266 
267                         lcd_clear();
268                         lcd_puts("Enter Password:");
269                     }
270                 }
```

۳ تصـــویر آخری مال شـــرایطی بودند که کاربر در حال تغییر رمز نبود و پس از وارد کردن رمز = را فشـــرده بود، حال اگر در حین وارد کردن رمز دو کلید – یا on/c را بزنند یا هنوز ۴ اعداد رمز را به طور کامل وارد نکرده باشد، دقیقا همان روندی تکرار می‌شود که پیش تر در مورد این کلیدهای بخصوص گفته شده بود.

```
272             else if (result == 10) {  // on/c - Clear all input
273                 pl_len = 0;
274                 pl[0] = '\0';
275                 display_input();
276             }
277             else if (result == 14) {  // C - Backspace (delete last digit)
278                 if (pl_len > 0) {
279                     pl_len--;
280                     pl[pl_len] = '\0';
281                 }
282                 display_input();
283             }
284             else if (result <= 9 && pl_len < 4) {
285                 pl[pl_len++] = '0' + result;
286                 pl[pl_len] = '\0';
287                 display_input();
288             }
289
290             delay(30);  // Debounce
291         }
292
293         delay(1);
294     }
295 }
```